МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП’ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ

до лабораторної роботи №3

з дисципліни «Програмування систем на кристалі»

Студента 4-го курсу
спеціальності 123 –
«Комп’ютерна інженерія»
Сеника Ярослава

м. Ужгород – 2026 р.

ЛАБОРАТОРНА РОБОТА №3

Тема: Використання матричної клавіатури для введення інформації у інтегрованому середовищі PSoC® Creator TM

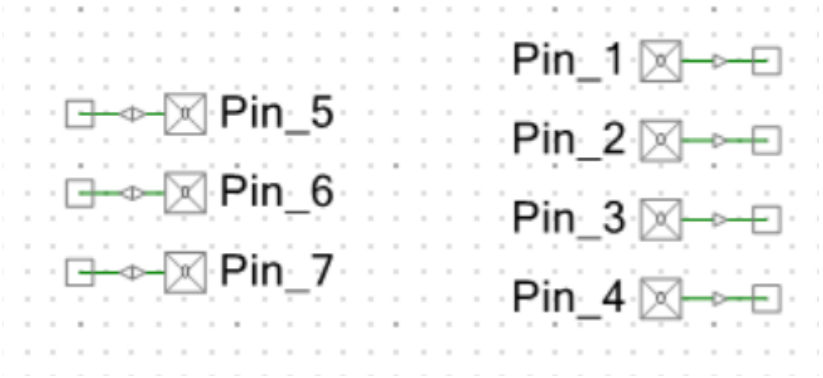
Мета: Ознайомитися з принципами підключення та використання матричної клавіатури для введення інформації у мікроконтролерних системах на базі PSoC. Набути навичок розробки проєкту в інтегрованому середовищі PSoC® CreatorTM, налаштування апаратних компонентів та програмної обробки введених даних.

Хід роботи

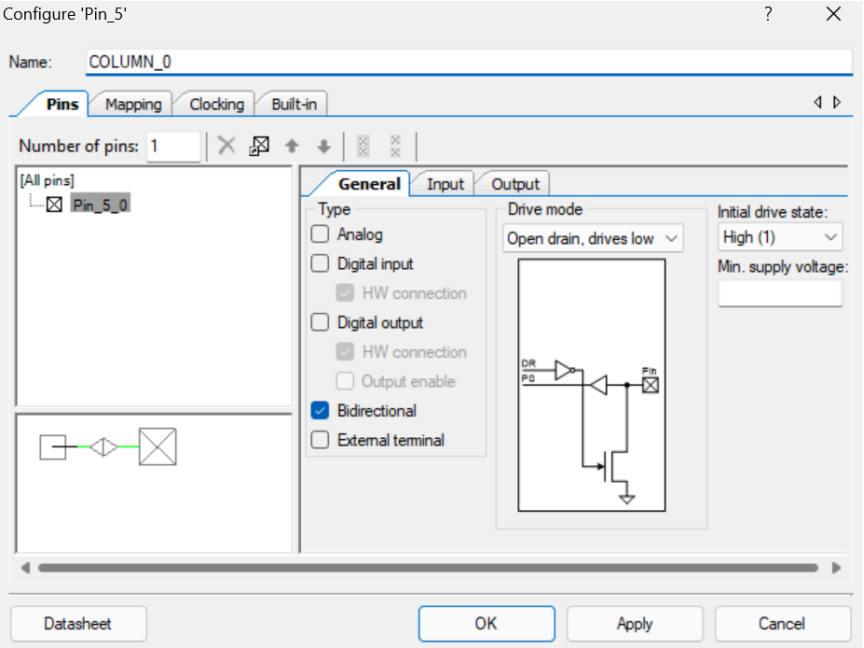
Завдання 1.

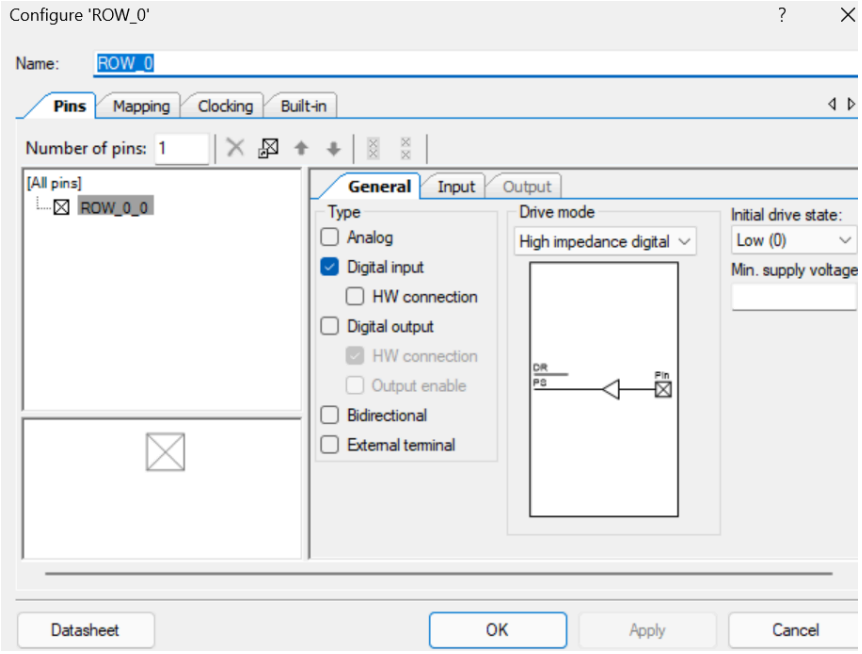
1. Скопіюйте проєкт Лабораторної роботи No2.

2. Перейдіть на вкладку TopDesighn.cysch. З каталогу Cypress додайте на схему три Двонаправлені піни (Digital Bidirectional Pin), які будемо використовувати в якості стовпців (Column) та чотири піни типу «Вхід» (Digital Input Pin), що будуть відповідати рядкам (Row) нашої матричної клавіатури

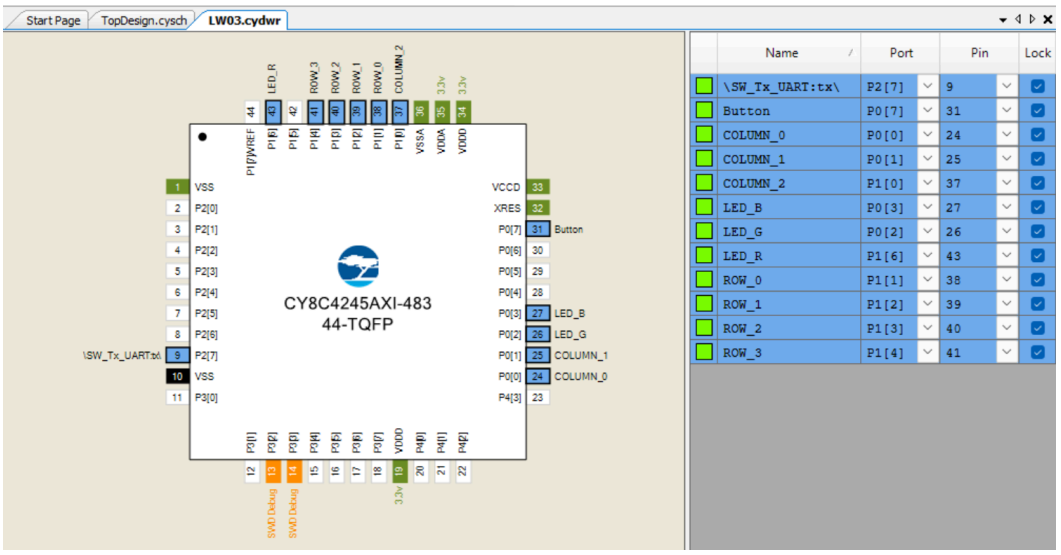


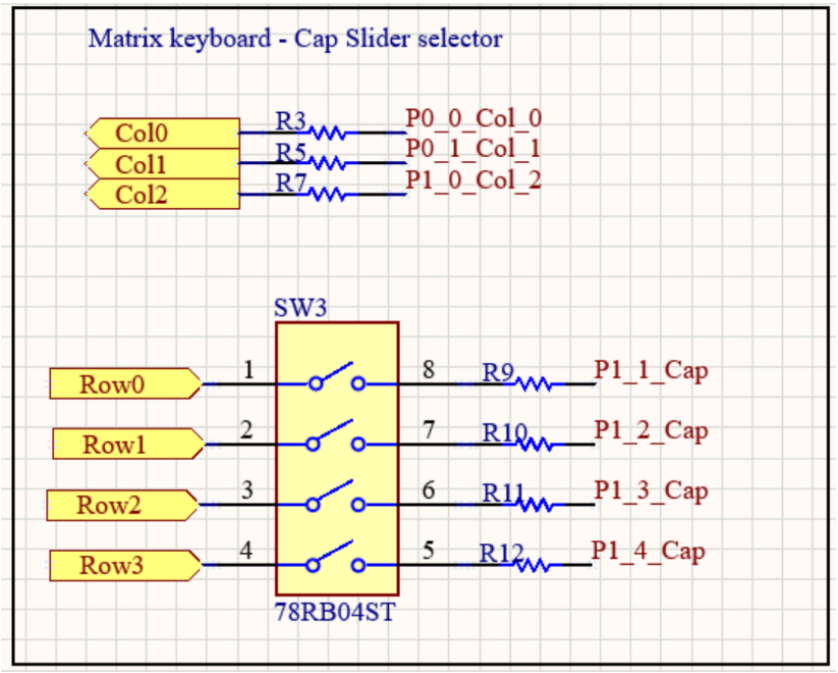
3. Після цього виконаємо налаштування Двонаправлених пінів Digital Bidirectional та пінів, типу «Вхід» Digital Input:



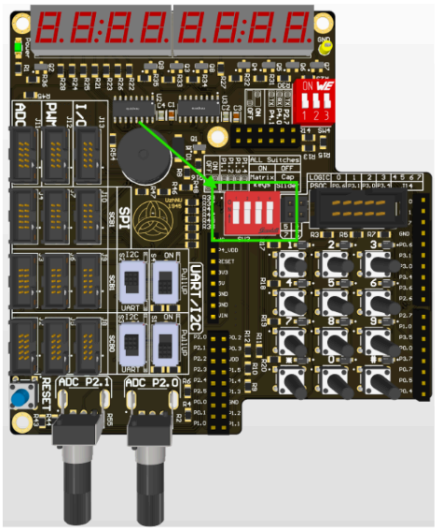


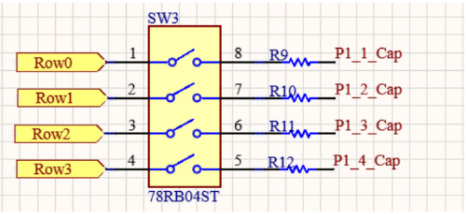
3. Перейдіть на вкладку Pins та підключіть рядки та стовпці в відповідності до схеми пінів на платі.





4. За посиланням можна ознайомитись зі схемою Шилда для PSoC Pioneer Kit. Для використання матричної клавіатури перемикаємо всі перемикачі SW3 в положення ON. Під час використання матричної клавіатури слайдер CapSence використовувати не вийде.





5. Напишіть код для опрацювання матричної клавіатури.

Для зручності опрацювання матричної клавіатури створюємо масиви вказівників на функції конфігурації, запису та читання піна. Також, необхідно оголосити масив фактичних значень кнопок (матриця станів кнопок та масив відповідності). Написати функцію ініціалізації, щоб перевести всі вихідні піни в непідключений стан (HIZ). Написати функцію опрацювання кнопок. Додати ініціалізацію та вивід значень в main().

Код:

```
include "project.h"

    /* arrays of pointers */
    /* function of drive mode configuration */
    static void (*COLUMN_x_SetDriveMode[3])(uint8_t mode) = {
        COLUMN_0_SetDriveMode,
        COLUMN_1_SetDriveMode,
        COLUMN_2_SetDriveMode
    };

    /* column write function */
    static void (*COLUMN_x_Write[3])(uint8_t value) = {
        COLUMN_0_Write,
        COLUMN_1_Write,
        COLUMN_2_Write
    };

    /* read row function */
    static uint8 (*ROW_x_Read[4])() = {
        ROW_0_Read,
        ROW_1_Read,
        ROW_2_Read,
```

```
        ROW_3_Read
    };

    /* [ROW][COLUMN] */
    static uint8_t keys[4][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9},
        {10, 0, 11},
    };

    /* matrix initialization function */
    static void initMatrix()
    {
            for(int  column_index  =  0;  column_index  <  3;
column_index++)
        {

COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
        }
    }

    /* keys matrix read function */
    static void readMatrix()
    {
        /* define the length of a row and column */
                                uint8_t       row_counter       =
sizeof(ROW_x_Read)/sizeof(ROW_x_Read[0]);
                              uint8_t      column_counter      =
sizeof(COLUMN_x_Write)/sizeof(COLUMN_x_Write[0]);

        /* column: iterate the columns */
         for(int column_index = 0; column_index < column_counter;
column_index++)
        {

COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_STRONG);
            COLUMN_x_Write[column_index](0);

            /* row: interate throught the rows */
                for(int  row_index  =  0;  row_index  <  row_counter;
row_index++)
            {
                                  keys[row_index][column_index]  =
ROW_x_Read[row_index]();
            }

            /* disable the column */

COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
        }
    }
```

```
int main(void)
{
    CyGlobalIntEnable; /* Enable global interrupts. */

    SW_Tx_UART_Start();
    SW_Tx_UART_PutCRLF();
    SW_Tx_UART_PutString("Software Transmit UART");
    SW_Tx_UART_PutCRLF();

    initMatrix();
    uint8_t last_state = 0;

    for(;;)
    {
        /* Button == 0 -> Button pressed
            Button == 1 -> Button released
            LED_x_Write(0); -> Turn ON
            LED_x_Write(1); -> Turn OFF
        */
        readMatrix();

          if(keys[0][0] == 0 && last_state != 1) //is button
pressed
        {
            last_state = 1;
            SW_Tx_UART_PutString("Button 1 pressed");
            SW_Tx_UART_PutCRLF();
            LED_R_Write(0); // Turn ON
            LED_G_Write(0);
            LED_B_Write(0);
        }

          if(keys[0][1] == 0 && last_state != 2) //is button
pressed
        {
            last_state = 2;
            SW_Tx_UART_PutString("Button 2 pressed");
            SW_Tx_UART_PutCRLF();
            LED_R_Write(0); // Turn ON
            LED_G_Write(0);
            LED_B_Write(0);
        }

          if(keys[0][2] == 0 && last_state != 3) //is button
pressed
        {
            last_state = 3;
            SW_Tx_UART_PutString("Button 3 pressed");
            SW_Tx_UART_PutCRLF();
            LED_R_Write(0); // Turn ON
            LED_G_Write(0);
            LED_B_Write(0);
        }
```

```
                if(keys[1][0] == 0 && last_state != 4) //is button
pressed
            {
                last_state = 4;
                SW_Tx_UART_PutString("Button 4 pressed");
                SW_Tx_UART_PutCRLF();
                LED_R_Write(0); // Turn ON
                LED_G_Write(0);
                LED_B_Write(0);
            }

                if(keys[1][1] == 0 && last_state != 5) //is button
pressed
            {
                last_state = 5;
                SW_Tx_UART_PutString("Button 5 pressed");
                SW_Tx_UART_PutCRLF();
                LED_R_Write(0); // Turn ON
                LED_G_Write(0);
                LED_B_Write(0);
            }

                if(keys[1][2] == 0 && last_state != 6) //is button
pressed
            {
                last_state = 6;
                SW_Tx_UART_PutString("Button 6 pressed");
                SW_Tx_UART_PutCRLF();
                LED_R_Write(0); // Turn ON
                LED_G_Write(0);
                LED_B_Write(0);
            }

                if(keys[2][0] == 0 && last_state != 7) //is button
pressed
            {
                last_state = 7;
                SW_Tx_UART_PutString("Button 7 pressed");
                SW_Tx_UART_PutCRLF();
                LED_R_Write(0); // Turn ON
                LED_G_Write(0);
                LED_B_Write(0);
            }

                if(keys[2][1] == 0 && last_state != 8) //is button
pressed
            {
                last_state = 8;
                SW_Tx_UART_PutString("Button 8 pressed");
                SW_Tx_UART_PutCRLF();
                LED_R_Write(0); // Turn ON
                LED_G_Write(0);
```

```
                LED_B_Write(0);
            }

              if(keys[2][2] == 0 && last_state != 9) //is button
pressed
            {
                last_state = 9;
                SW_Tx_UART_PutString("Button 9 pressed");
                SW_Tx_UART_PutCRLF();
                LED_R_Write(0); // Turn ON
                LED_G_Write(0);
                LED_B_Write(0);
            }

             if(keys[3][0] == 0 && last_state != 10) //is button
pressed
            {
                last_state = 10;
                SW_Tx_UART_PutString("Button * pressed");
                SW_Tx_UART_PutCRLF();
                LED_R_Write(0); // Turn ON
                LED_G_Write(0);
                LED_B_Write(0);
            }

              if(keys[3][1] == 0 && last_state != 0) //is button
pressed
            {
                last_state = 0;
                SW_Tx_UART_PutString("Button 0 pressed");
                SW_Tx_UART_PutCRLF();
                LED_R_Write(0); // Turn ON
                LED_G_Write(0);
                LED_B_Write(0);
            }

             if(keys[3][2] == 0 && last_state != 11) //is button
pressed
            {
                last_state = 11;
                SW_Tx_UART_PutString("Button # pressed");
                SW_Tx_UART_PutCRLF();
                LED_R_Write(0); // Turn ON
                LED_G_Write(0);
                LED_B_Write(0);
            }
        }
    }
```

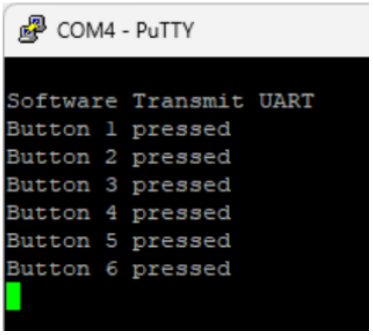
6. Скомпілювати та запрограмувати проєкт в PSoC® 4 Pioneer Kit.

Build → Compile File (Ctrl+F6)

Build → Build (Shift+F6)

Debug → Program (Ctrl+F5)

7. Відкрити термінальну програму Putty. Натиснути на кнопки клавіатури та проаналізувати вивід в терміналі.

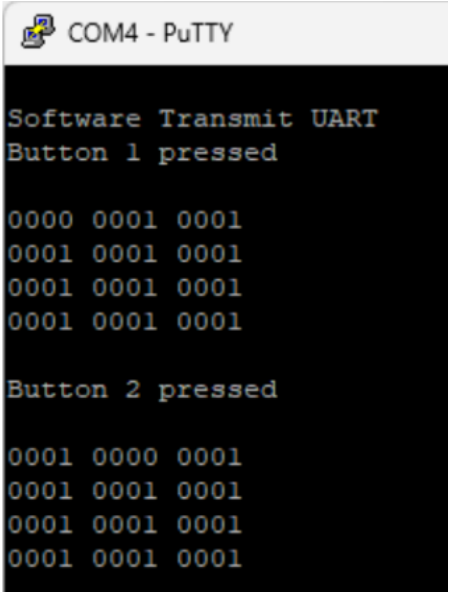


8. Добавити вивід стану матриці.

Код:

```
/* matrix print function */
     void printMatrix()
     {
         SW_Tx_UART_PutCRLF();
         for (int row_index = 0; row_index < 4; row_index++)
         {
                    for(int  column_index  =  0;  column_index  <  3;
column_index++)
             {
SW_Tx_UART_PutHexInt(keys[row_index][column_index]);
                 SW_Tx_UART_PutString(" ");
             }
             SW_Tx_UART_PutCRLF();
         }
         SW_Tx_UART_PutCRLF();
     }
```

9. Самостійно модифікувати main() з використанням цієї функції для виводу стану матричної клавіатури.



10. Самостійно додати вивід натиснутої кнопки в термінал на прикладі Лабораторної роботи No2. В термінал повинна виводитись подія «Button pressed» та подія «Button released».

Лістинг коду:

```
/* ------------------ КНОПКА 1 ------------------ */
        if(keys[0][0] == 0 && last_state != 1)
        {
            last_state = 1;
            SW_Tx_UART_PutString("Button 1 pressed");
            SW_Tx_UART_PutCRLF();
            LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
        }
        else if (keys[0][0] == 1 && last_state == 1)
        {
            last_state = 99;
            SW_Tx_UART_PutString("Button 1 released");
            SW_Tx_UART_PutCRLF();
            LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
        }

        /* ------------------ КНОПКА 2 ------------------ */
        if(keys[0][1] == 0 && last_state != 2)
        {
            last_state = 2;
            SW_Tx_UART_PutString("Button 2 pressed");
            SW_Tx_UART_PutCRLF();
            LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
        }
        else if (keys[0][1] == 1 && last_state == 2)
        {
            last_state = 99;
            SW_Tx_UART_PutString("Button 2 released");
```

```
        SW_Tx_UART_PutCRLF();
        LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
    }

    /* ------------------ КНОПКА 3 ------------------ */
    if(keys[0][2] == 0 && last_state != 3)
    {
        last_state = 3;
        SW_Tx_UART_PutString("Button 3 pressed");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
    }
    else if (keys[0][2] == 1 && last_state == 3)
    {
        last_state = 99;
        SW_Tx_UART_PutString("Button 3 released");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
    }

    /* ------------------ КНОПКА 4 ------------------ */
    if(keys[1][0] == 0 && last_state != 4)
    {
        last_state = 4;
        SW_Tx_UART_PutString("Button 4 pressed");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
    }
    else if (keys[1][0] == 1 && last_state == 4)
    {
        last_state = 99;
        SW_Tx_UART_PutString("Button 4 released");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
    }

    /* ------------------ КНОПКА 5 ------------------ */
    if(keys[1][1] == 0 && last_state != 5)
    {
        last_state = 5;
        SW_Tx_UART_PutString("Button 5 pressed");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
    }
    else if (keys[1][1] == 1 && last_state == 5)
    {
        last_state = 99;
        SW_Tx_UART_PutString("Button 5 released");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
    }

    /* ------------------ КНОПКА 6 ------------------ */
```

```
if(keys[1][2] == 0 && last_state != 6)
{
    last_state = 6;
    SW_Tx_UART_PutString("Button 6 pressed");
    SW_Tx_UART_PutCRLF();
    LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
}
else if (keys[1][2] == 1 && last_state == 6)
{
    last_state = 99;
    SW_Tx_UART_PutString("Button 6 released");
    SW_Tx_UART_PutCRLF();
    LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
}

/* ------------------ КНОПКА 7 ------------------ */
if(keys[2][0] == 0 && last_state != 7)
{
    last_state = 7;
    SW_Tx_UART_PutString("Button 7 pressed");
    SW_Tx_UART_PutCRLF();
    LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
}
else if (keys[2][0] == 1 && last_state == 7)
{
    last_state = 99;
    SW_Tx_UART_PutString("Button 7 released");
    SW_Tx_UART_PutCRLF();
    LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
}

/* ------------------ КНОПКА 8 ------------------ */
if(keys[2][1] == 0 && last_state != 8)
{
    last_state = 8;
    SW_Tx_UART_PutString("Button 8 pressed");
    SW_Tx_UART_PutCRLF();
    LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
}
else if (keys[2][1] == 1 && last_state == 8)
{
    last_state = 99;
    SW_Tx_UART_PutString("Button 8 released");
    SW_Tx_UART_PutCRLF();
    LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
}

/* ------------------ КНОПКА 9 ------------------ */
if(keys[2][2] == 0 && last_state != 9)
{
    last_state = 9;
    SW_Tx_UART_PutString("Button 9 pressed");
    SW_Tx_UART_PutCRLF();
```

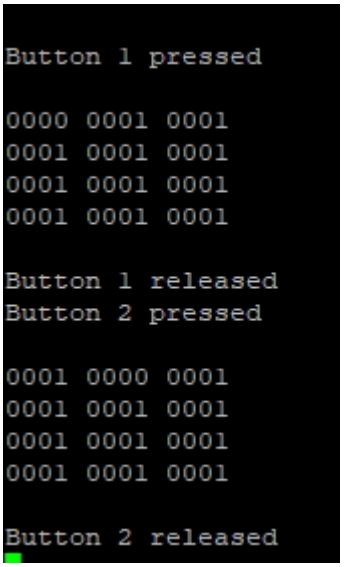
```
        LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
    }
    else if (keys[2][2] == 1 && last_state == 9)
    {
        last_state = 99;
        SW_Tx_UART_PutString("Button 9 released");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
    }

    /* ------------------ КНОПКА * ------------------ */
    if(keys[3][0] == 0 && last_state != 10)
    {
        last_state = 10;
        SW_Tx_UART_PutString("Button * pressed");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
    }
    else if (keys[3][0] == 1 && last_state == 10)
    {
        last_state = 99;
        SW_Tx_UART_PutString("Button * released");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
    }

    /* ------------------ КНОПКА 0 ------------------ */
    if(keys[3][1] == 0 && last_state != 0)
    {
        last_state = 0;
        SW_Tx_UART_PutString("Button 0 pressed");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
    }
    else if (keys[3][1] == 1 && last_state == 0)
    {
        last_state = 99;
        SW_Tx_UART_PutString("Button 0 released");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
    }

    /* ------------------ КНОПКА # ------------------ */
    if(keys[3][2] == 0 && last_state != 11)
    {
        last_state = 11;
        SW_Tx_UART_PutString("Button # pressed");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(0); LED_G_Write(0); LED_B_Write(0);
    }
    else if (keys[3][2] == 1 && last_state == 11)
    {
        last_state = 99;
```

```
        SW_Tx_UART_PutString("Button # released");
        SW_Tx_UART_PutCRLF();
        LED_R_Write(1); LED_G_Write(1); LED_B_Write(1);
    }
```



Завдання 2.

1. На початку, коли всі кнопки відпущені горить колір White. Запрограмувати кнопки так, щоб при натиску на кнопку світлодіод змінював колір за певним алгоритмом. Після натиску, коли кнопка відпущена світлодіод вимикається (Black).

Таблиця 1 - Кольори кнопок

Red	1, 7
Green	2, 8
Blue	3, 9
Yellow	4, *
Purple	5, 0
Cyan	6, #

2. Реалізувати даний проєкт переписавши код програми під використання конструкції Swich(case).

3. Встановити пароль (довжиною > 2 символів). Якщо користувач ввів пароль вірнотт в консоль виводиться повідомлення «Access allowed», в іншому випадку «Access denied».

Виконання завдання 2:

Код:

```
#include "project.h"

/* Масиви вказівників на функції для керування пінами */
static void (*COLUMN_x_SetDriveMode[3])(uint8 mode) = {
    COLUMN_0_SetDriveMode,
    COLUMN_1_SetDriveMode,
    COLUMN_2_SetDriveMode
};

static void (*COLUMN_x_Write[3])(uint8 value) = {
    COLUMN_0_Write,
    COLUMN_1_Write,
    COLUMN_2_Write
};

static uint8 (*ROW_x_Read[4])(void) = {
    ROW_0_Read,
    ROW_1_Read,
    ROW_2_Read,
    ROW_3_Read
};

/* Масив для зберігання стану кнопок [ROW][COLUMN] */
static uint8 keys[4][3] = {
    {1, 1, 1},
    {1, 1, 1},
    {1, 1, 1},
    {1, 1, 1}
};

/* Масив для ідентифікації кнопок: 10 - це '*', 11 - це '#' */
static const uint8 button_ids[4][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
    {10, 0, 11}
};

/* Функція ініціалізації матриці */
static void initMatrix(void)
{
    uint8 column_index;
    for(column_index = 0; column_index < 3; column_index++)
    {
        COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

/* Функція зчитування стану матриці */
static void readMatrix(void)
{
```

```
    uint8 row_counter = 4;
    uint8 column_counter = 3;
    uint8 column_index, row_index;

         for(column_index  =  0;  column_index  <  column_counter;
column_index++)
    {
        COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_STRONG);
        COLUMN_x_Write[column_index](0);

        for(row_index = 0; row_index < row_counter; row_index++)
        {
                                keys[row_index][column_index]   =
ROW_x_Read[row_index]();
        }

        COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

int main(void)
{
    /* Оголошення змінних */
    uint8 r, c;
      uint8  last_state  =  99;  //  99  означає,  що  жодна  кнопка  не
натиснута
    uint8 current_pressed;

    // Змінні для пароля
    uint8 input_pass[4] = {0, 0, 0, 0};
    uint8 pass_index = 0;
      uint8  correct_pass[4]  =  {1,  2,  3,  4};  //  Еталонний  пароль
(1-2-3-4)

    // ПРАПОРЕЦЬ АВТОРИЗАЦІЇ: 0 - заблоковано, 1 - розблоковано
    uint8 is_logged_in = 0;

    CyGlobalIntEnable; /* Enable global interrupts. */

    SW_Tx_UART_Start();
    SW_Tx_UART_PutCRLF();
            SW_Tx_UART_PutString("System   Locked.   Enter   4-digit
password:");
    SW_Tx_UART_PutCRLF();

    initMatrix();

     /* Початковий стан: система заблокована, світлодіоди вимкнені
(Black) */
    LED_R_Write(1);
    LED_G_Write(1);
    LED_B_Write(1);
```

```
    for(;;)
    {
        readMatrix();
        current_pressed = 99;

        /* Скануємо матрицю, щоб знайти натиснуту кнопку */
        for(r = 0; r < 4; r++)
        {
            for(c = 0; c < 3; c++)
            {
                if(keys[r][c] == 0)
                {
                    current_pressed = button_ids[r][c];
                }
            }
        }

        /* Якщо кнопку ЩОЙНО натиснули */
        if(current_pressed != 99 && last_state == 99)
        {
            last_state = current_pressed;

            if (is_logged_in == 0)
            {
                // ==========================================
                // ЗАВДАННЯ 3: РЕЖИМ ВВОДУ ПАРОЛЯ
                // ==========================================
                input_pass[pass_index] = current_pressed;
                pass_index++;

                SW_Tx_UART_PutString("*");

                if(pass_index == 4)
                {
                    SW_Tx_UART_PutCRLF();

                    if(input_pass[0] == correct_pass[0] &&
                       input_pass[1] == correct_pass[1] &&
                       input_pass[2] == correct_pass[2] &&
                       input_pass[3] == correct_pass[3])
                    {
                              SW_Tx_UART_PutString("Access allowed!
System Unlocked.");
                         is_logged_in = 1; // РОЗБЛОКОВУЄМО СИСТЕМУ

                           // Згідно з завданням 1, на початку після
розблокування горить White
                                      LED_R_Write(0); LED_G_Write(0);
LED_B_Write(0);
                    }
                    else
                    {
```

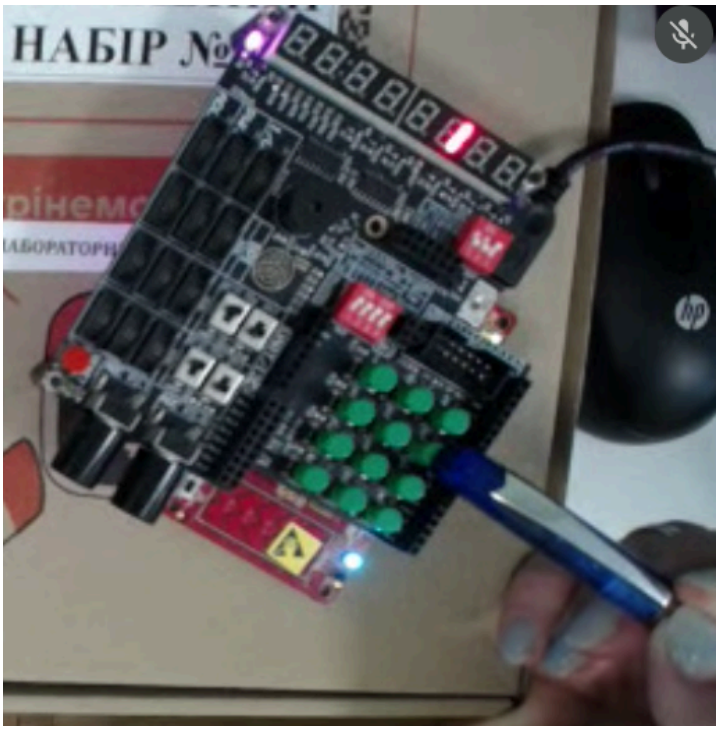
```
                            SW_Tx_UART_PutString("Access denied! Try
again:");
                        }

                        SW_Tx_UART_PutCRLF();
                        pass_index = 0; // Скидаємо лічильник
                    }
                }
                else
                {
                    // ==========================================
                    // ЗАВДАННЯ 1 і 2: РЕЖИМ КЕРУВАННЯ КОЛЬОРАМИ
                    // (Працює тільки після правильного пароля)
                    // ==========================================
                    switch(current_pressed)
                    {
                        case 1: case 7: // Red
                                        LED_R_Write(0);  LED_G_Write(1);
LED_B_Write(1); break;
                        case 2: case 8: // Green
                                        LED_R_Write(1);  LED_G_Write(0);
LED_B_Write(1); break;
                        case 3: case 9: // Blue
                                        LED_R_Write(1);  LED_G_Write(1);
LED_B_Write(0); break;
                        case 4: case 10: // Yellow (10 = *)
                                        LED_R_Write(0);  LED_G_Write(0);
LED_B_Write(1); break;
                        case 5: case 0: // Purple
                                        LED_R_Write(0);  LED_G_Write(1);
LED_B_Write(0); break;
                        case 6: case 11: // Cyan (11 = #)
                                        LED_R_Write(1);  LED_G_Write(0);
LED_B_Write(0); break;
                        default: break;
                    }
                }

                CyDelay(50); // Антибрязкіт
            }
            /* Якщо кнопку ЩОЙНО відпустили */
            else if(current_pressed == 99 && last_state != 99)
            {
                last_state = 99; // Скидаємо стан

                 // Вимикаємо світлодіод (Black) при відпусканні ТІЛЬКИ
якщо ми розблоковані
                if (is_logged_in == 1)
                {
                    LED_R_Write(1);
                    LED_G_Write(1);
                    LED_B_Write(1);
                }
```

```
                CyDelay(50); // Антибрязкіт
            }
        }
}
```



Висновки: Ознайомився з принципами підключення та використання матричної клавіатури для введення інформації у мікроконтролерних системах на базі PSoC. Набув навичок розробки проєкту в інтегрованому середовищі PSoC® CreatorTM, налаштування апаратних компонентів та програмної обробки введених даних.